

What a Student Benchmark Taught Me About Measuring Neural Network Speed

Aryan Kulkarni

Bellarmino College Preparatory, San Jose, CA, USA

aryanamol10@gmail.com

Abstract—As a high-school researcher, I compared sequential, parallel, modular, and hybrid neural networks on three synthetic regression tasks. My first benchmark suggested that modular networks train $1.62\times$ faster. An audit showed that this came from mixing two software frameworks and from inputs that carried no information. A corrected, parameter-matched study with five random seeds found that how parameters are wired changes inference latency far more than parameter count does, and that a modular design matched to the problem’s structure gave the best accuracy on the hardest task. I describe the project as a case study in experimental rigor for pre-college computing research.

I. Motivation

Neural networks are often compared by parameter count, but users experience latency. I asked: with the same parameter budget, does arranging the parameters in sequence, in parallel, or as modules change speed and accuracy?

II. From a Flawed Pilot to a Fair Test

The pilot used Keras for two models and PyTorch for the third. The “fast” modular model was mathematically identical to the sequential one, so the speedup was the framework’s, not the architecture’s. Its training inputs were also random numbers unrelated to the targets, so every model ended up predicting the average. The lesson I took away was to change one variable at a time, hold out test data, repeat with several seeds, and include a control that should show no difference.

III. Main Study

Six models (about 385 parameters each) were trained in PyTorch on linear, curved, and chained functions, with five seeds each. A latency sweep timed every model at batch sizes from 1 to 4096.

IV. Findings

- On the chained task, the staged modular hybrid (H-B) reached test MSE 0.326, compared with 0.571 for the plain sequential network.
- H-B was also the slowest model at batch size 1 (38.2 μ s vs. 11.7 μ s), because it runs the most operations per call.
- Parameter count alone predicted neither speed nor accuracy.

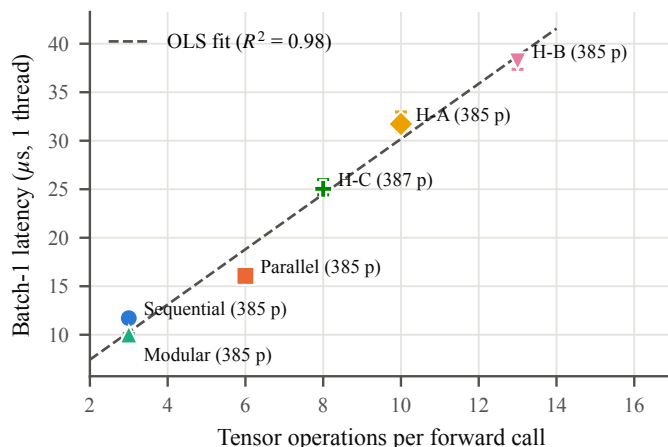


Fig. 1. Single-sample latency grows with the number of operations per call ($R^2 = 0.98$), even though every model has about the same number of parameters.

V. Reflection for STEM Education

The surprise. My first benchmark seemed to show that modular networks were $1.62\times$ faster, and I [was ready to write it up / believed it at first]. The moment I realized it was wrong was [describe it: e.g., noticing the “fast” model was mathematically identical to the slow one]. [How did that feel, and what did you do next?]

The lesson. I learned that a fair comparison changes only one thing at a time. Mixing two software frameworks meant I was measuring the frameworks, not the architectures. Now I [plan a control that should show no difference / write down what I expect before running].

The process. The hardest new skill was [e.g., running five random seeds and reporting the spread / holding out test data / reading the PyTorch source]. It mattered because [one sentence].

For other students. If I could tell another student one thing, it would be: [your advice]. Projects like this need no special hardware; every model here trains on a laptop.

Acknowledgment

The pilot study used Google Colab. AI tools assisted with experiment code, figures, and drafting; the author verified all content.